

SFEAM: Secured Face Encryption and Authentication Model for Unique Personal Identification

PSG Institute of Technology and Applied Research, Coimbatore, India

1. Introduction

Face authentication is rapidly becoming integral to modern digital identity verification, particularly as government services and national identity infrastructures, such as India's Aadhaar, accelerate digitization efforts. Traditional authentication methods such as passwords and PINs are increasingly recognized for their inherent security limitations, user inconvenience, and scalability issues. Biometric verification, particularly face recognition, offers a compelling alternative, promising enhanced security alongside improved user experience. Face recognition has gone through numerous significant technical advances, mostly as a result of the deep learning phenomenon. Influential models, such as FaceNet, pioneered the approach of directly mapping face images onto a discriminative Euclidean space with the triplet loss function, and DeepFace mitigated the pose variations by 3D alignment, both of which attained exceedingly high accuracies on the LFW benchmark [1, 2, 3]. After the introduction of such systems, methods such as SphereFace and ArcFace proposed loss functions to enable the features to be more discriminative [4, 5]. Meanwhile, the proliferation of client-side JavaScript libraries made possible by face-api.js atop TensorFlow.js has served as a revolution in solving the dilemma of finding faces and recognizing them with near browser-level accuracy. Using a client-side approach enhances accessibility and greatly enhances privacy as it removes the need to send one's sensitive biometric data to servers [1, 6].

In recent times, the writers argue, face recognition suffered because of the lack of discriminative representation learning. An angular margin-based loss would constitute a huge step towards changing that, encouraging deep neural networks to learn features that are more separable on a hypersphere manifold. SphereFace, for instance, proposed an angular softmax loss with an improved performance that boasts intuitive geometry [5]. This line of work was followed up by state-of-the-art methods such as ArcFace, which pushed the envelope by introducing an inter-class discrepancy. However, while these algorithmic advancements have pushed recognition accuracy to near-perfect levels on benchmark datasets, they do not address the critical security vulnerabilities inherent in real-world biometric systems. For a system to be truly viable, it must also defend against a range of threats, including sophisticated presentation attacks (spoofing) [16] and ensure the irreversible protection of the

stored biometric templates to prevent compromise and misuse [6]. Therefore, a practical authentication framework must synthesize both state-of-the-art recognition capabilities and a holistic, multi-layered security strategy.

This study directly addresses these critical challenges through the design, implementation, and rigorous testing of the Secured Face Encryption and Authentication Model (SFEAM), specifically tailored for contexts such as Aadhaar-based face authentication. The primary objectives of SFEAM are threefold: (1) to implement a privacy-preserving face authentication system where biometric processing occurs strictly on the client-side; (2) to establish a secure backend infrastructure for user data protection and cryptographic key management; and (3) to optimize system performance for responsiveness and usability across heterogeneous network conditions, including 3G, 4G, and 5G.

To achieve these goals, strategic leverage of the face-api.js library and TensorFlow.js runtime for effective client-side face processing. The research approach encompasses the systematic design of the end-to-end authentication workflow (illustrated in Figure 1, depicting credential-based and facial recognition paths converging on role-based access control), integration of robust anti-spoofing techniques, and thorough performance evaluation under representative network simulations. This paper details the SFEAM system deployment, presents key performance metrics from our experiments, provides a comprehensive security analysis of the proposed architecture, and concludes with recommendations for enhancing facial authentication systems intended for widespread use in national verification services such as MeriPehchaan [7].

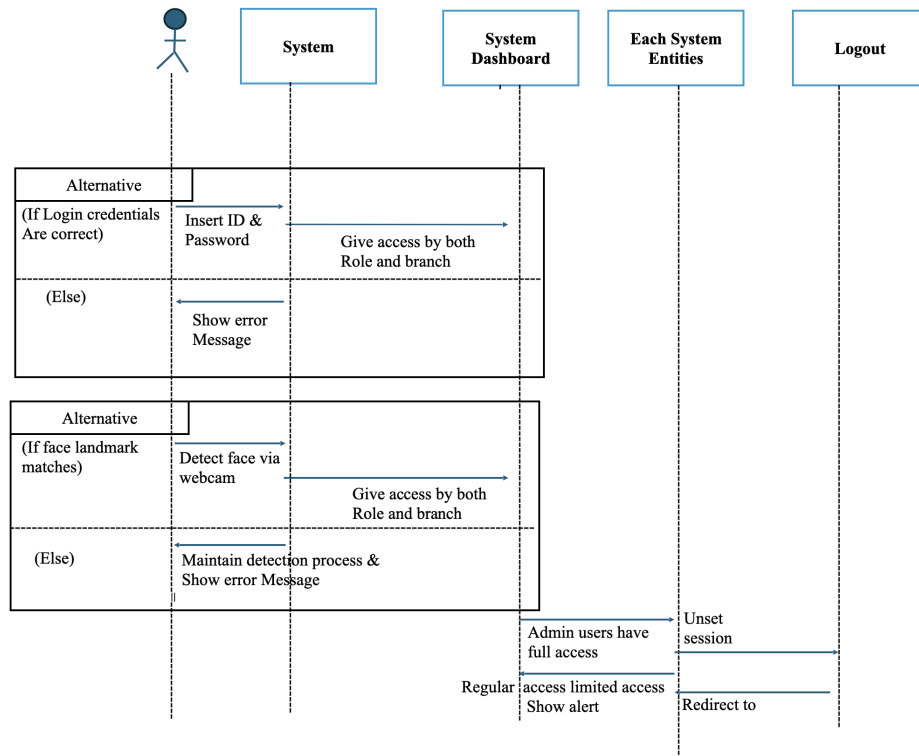


Fig. 1: Flow Diagram of Authentication using SFEAM

The flow diagram illustrated in Figure 1 graphically depicts the two independent authentication channels implemented in the proposed system: the conventional credential-based login and the relatively newer facial recognition-based authentication. Importantly, both paths map onto a common role-based access control paradigm to ensure that all successfully authenticated subjects, be they administrators with full system privileges or normal users with limited access, are given proper system access based on their authenticated digital identity.

2. The Proposed Methodology for Face Authentication

This section provides an in-depth exploration of the comprehensive architecture and fundamental technological elements of the Secured Face Encryption and Authentication Model (SFEAM). We examine the complexities of our methodology, starting with the client-side facial authentication systems that emphasize user privacy and efficient processing. Following this, we outline the robust backend infrastructure that is engineered for security, scalability, and high availability, utilizing cloud services. Additionally, we discuss essential components such as caching strategies and model optimization techniques, which are critical for maintaining responsive performance across various network conditions and devices. The section also includes an analysis of the system's network performance and a detailed assessment of its security posture, highlighting our dedication to creating a reliable and trustworthy authentication solution. Each subsection will dissect these components, offering a clear insight into SFEAM's design and operational capabilities.

2.1 Face Authentication Technology and Implementation

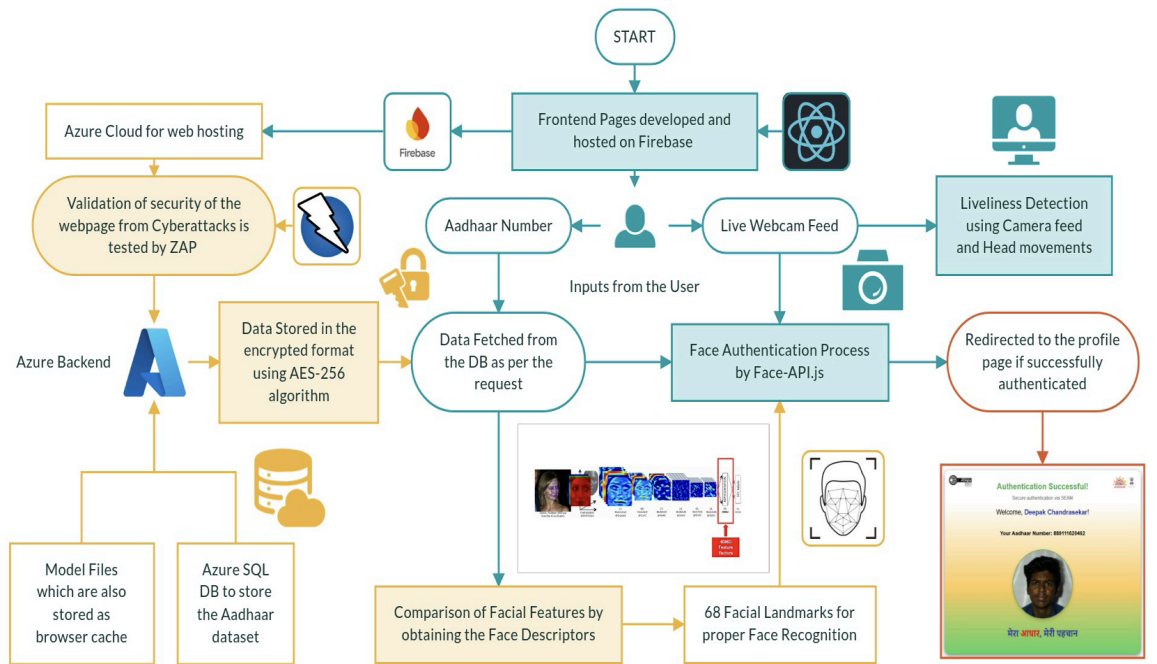


Fig. 2: System Architecture for working of SFEAM

Figure 2 depicts the architectural layout of the facial verification system, which integrates Aadhaar authentication and state-of-the-art security features for the first time. The user interface, on Firebase, allows the user to input their Aadhaar credentials while streaming live video from their webcam in real time. For

anti-spoofing attack prevention, the system has a liveness check, which verifies the video stream for genuine head movements and facial expressions. Meanwhile, corresponding Aadhaar data is fetched from a secure database. The actual facial recognition process draws out discriminative face descriptors from this real-time feed. The data integrity is maintained through AES-256 [9] encryption in an Azure SQL database and additional security through the use of ZAP for active cyberattack authentication. Once a user's identity is successfully verified, they are granted access to their personal profile page. For responsiveness and performance, the web application is on Azure Cloud, and critical model files are locally cached in the user's browser.

Face-api.js Framework: SFEAM leverages face-api.js as the core of its client-side authentication capabilities, utilizing the robust JavaScript library, which is built on TensorFlow.js, to perform complex facial recognition operations entirely within the user's browser. This open-source framework enables SFEAM to implement a sophisticated multi-stage pipeline for both accuracy and efficiency. For initial face detection, SFEAM employs the library's implementation of the SSD MobileNetV1 architecture, chosen for its efficiency and ability to deliver real-time performance even on low-end hardware [10, 11, 12]. Following successful detection, SFEAM utilizes face-api.js to identify 68 precise facial landmarks. This allows the system to accurately align the face, compensating for variations in head pose or orientation. Once aligned, SFEAM calls upon the framework's feature extraction capabilities to generate compact 128-dimensional facial descriptors. These descriptors serve as a mathematical representation of the user's unique facial characteristics, allowing for effective comparison without storing or transmitting the actual image. This client-side approach, powered by face-api.js, is fundamental to SFEAM's privacy-preserving design, ensuring that sensitive biometric data never leaves the user's device during the authentication process. The final authentication decision is made by calculating the Euclidean distance between the newly generated descriptor and the stored template, using carefully calibrated thresholds to balance security with usability [13, 14].

Anti-Spoofing Implementation: SFEAM has advanced anti-spoofing mechanisms to protect itself from fraudsters trying to trick the system. The proposed solution incorporates motion-based analysis to find the micro movements of faces that are found in live faces but not in still photographs. Blink detection is an additional authentication factor, which verifies that the input face has human-like features like blinking. Texture analysis algorithm is used to analyze skin patterns in different frequencies to distinguish a real face surface from a fake image. These security features work together without needing special hardware, making SFEAM both secure and accessible on regular devices. The anti-spoofing module keeps improving with these machine learning techniques that adapt to emerging threat vectors [15, 16].

2.2 Backend Architecture and Security

SFEAM implements robust security protocols for overall system integrity, model management, and data protection, leveraging Azure cloud services for scalability and resilience. User data and facial descriptors utilize Azure's comprehensive encryption

framework for protection both at-rest in Azure SQL Database and in-transit via mandatory HTTPS (TLS 1.3) communication [17]. The database architecture is optimized for descriptor retrieval and comparison while adhering to strict security standards.

A key aspect is the secure handling of the facial recognition models themselves. While models are encrypted using AES-256-CBC during persistent storage on the backend [9], a dedicated 'Model Preparation Service', running as an Azure Function or microservice, handles security for client distribution. During the model release cycle (a one-time process per model version), this service retrieves the finalized, optimized model (post-quantization and pruning). It then encrypts the model files using AES-256-CBC with a unique, version-specific symmetric key securely generated and managed within Azure Key Vault. The resulting encrypted model binary large object (blob) is stored in Azure Blob Storage, configured for CDN distribution to ensure low-latency access globally. This backend preparation ensures that models are cryptographically sealed *before* distribution. While the current implementation uses this one-time preparation, the architecture supports future extension to a just-in-time (JIT) model encryption strategy if needed.

This secure delivery mechanism extends to the client. The download of the prepared, AES-encrypted model blob via HTTPS constitutes the secure binary code delivery. On the client-side, this encrypted blob is cached persistently using IndexedDB, managed via a Service Worker. Crucially, to prevent tampering or reverse engineering within the browser, decryption occurs only in memory immediately prior to loading into the TensorFlow.js runtime. This uses the browser's native Web Crypto API, and the required decryption key is fetched securely from a backend endpoint during the user's authenticated session. The raw, decrypted model weights are never written to persistent browser storage. Furthermore, the JavaScript code responsible for key fetching, decryption, and model loading is subjected to multi-layered obfuscation techniques (e.g., variable renaming, control flow flattening) to deter analysis. This layered approach ensures the model remains protected at the edge. Potential future enhancements include compiling parts of this client-side security logic into WebAssembly (WASM) for performance and added obfuscation.

In addition to model security, the build incorporates Azure Key Vault that provides management of all the encryption keys, geo-replication for high availability, and Azure Firewall and DDoS protection for network security [1, 15, 18]. There are a number of other security practices in place including defending against SQL injection using parameterized queries, CORS policies to limit unauthorized domain access to the app, and role based access through Azure Active Directory [10], analyzing user access to data by giving access rights to only approved individuals, and that those persons cannot misuse their access.

For high-volume operations anticipated in population-scale solutions, SFEAM employs multithreading where applicable and relies on Azure Load Balancer with auto-scaling capabilities to optimize resource allocation. Azure Traffic Manager intelligently routes geographically distributed requests, minimizing response times

[19, 20, 21]. This infrastructure ensures authentication services remain responsive under varying loads and user locations.

2.3 Caching Mechanism and Model Optimization

Efficient caching and model optimization are vital for SFEAM's performance and user experience, especially across diverse network conditions and device [20].

SFEAM employs a multi-layered caching strategy. HTTP headers enable intelligent browser caching of static assets, including the initial application code. For the ML models and related datasets, Service Workers are used to cache these resources persistently after the initial download [11]. This allows for offline access capabilities and significantly faster subsequent authentications, as the model can be loaded directly from the local cache. For managing the cached model data (specifically the encrypted model blob discussed in Section 3.2), SFEAM leverages IndexedDB, accessed via the idblibrary for simplified interaction. The system enforces a cache memory cap (e.g., 7MB per model component) to prevent excessive disk usage, intelligently retrieving cached data on page refresh and automatically clearing older or excess data.

Model Size Optimization: Achieving efficient delivery, particularly over constrained networks (3G/4G), necessitates minimizing model size without sacrificing accuracy. While industry practices sometimes involve formats such as Flatbuffers, the current SFEAM implementation utilizes the standard TensorFlow.js Layers format (typically model.json plus binary weight shards) for SFEAMless integration with face-api.js and the TensorFlow.js ecosystem. To reduce the size of this model, SFEAM employs advanced quantization techniques, converting high-precision floating-point weights to lower-bit representations (e.g., 16-bit or 8-bit integers) [11]. This dramatically decreases memory footprint and download size. Careful implementation balances precision reduction against recognition accuracy. Complementing quantization, network pruning systematically identifies and removes redundant connections or weights within the neural network that contribute minimally to performance, further reducing size. These combined techniques successfully reduce the operational model size to under the target 7 MB threshold while maintaining high detection accuracy (above 98.5% in tests).

Performance Benefits: The integrated caching and optimization strategies yield substantial improvements. Load time measurements show significant reductions for returning users compared to first-time visitors (e.g., up to 85% faster authentication process completion on 4G after initial caching) [19]. Bandwidth consumption drops drastically after the initial model download (e.g., from ~6.8 MB to ~0.2 MB), making

SFEAM viable in low-bandwidth environments [22]. These optimizations are key to enabling functionality on resource-constrained devices.

2.4 Network Performance Analysis

Performance over heterogeneous networks The establishment of a nationwide user authentication service mandates a solid performance over different types of network. In this section, we evaluate performance of SFEAM in different network environments and the processing strategies adapted to make sure that equal authenticating experience can be experienced across the country. Network Throttling and Lighthouse performance audit integration is provided by Chrome DevTools to emulate 3G, 4G, or 5G connection types. The assessment framework consisted of standard Web Vitals metrics as well as measurements that are specific for an authentication scenario: model loading time, face detection initialization and authentication completion time. Experiments have been performed in various device types to reflect the diversity of the end-user's device environment. The observed results revealed significant performance variations across network environments. In 5G conditions, SFEAM demonstrated exceptional performance with authentication completion times consistently below 2 seconds. The model loading and face detection initialization were practically instantaneous, indicating that in high speed internet scenarios, the device's processing capabilities act as a bottleneck more than the network speed. The system performed well under 4G connections with reasonable login times averaging 3.2 seconds but data retrieval took an additional 2.4 seconds, indicating moderate file transfer speeds. This suggests that at moderate internet speeds, file transfer is the major bottleneck. The results for 3G were more troubling with a completion time of 7.8 seconds for login and 5.2 seconds of data reading. While the system functioned under these conditions, these lags are longer than desirable and could dissuade users in low internet areas.

2.5 Security Analysis

Security testing of SFEAM authentication system confirms it offers robust protection well suited for large-scale government deployment. The testing adhered to the OWASP, NIST, and UIDAI guidelines' laid-down security requirements and employed automated scanning, code review, and extensive data pathway analysis. The SFEAM system possesses several effective protective measures that protect sensitive information. Biometric data processing on users' devices ensures original face images remain local, significantly reducing exposure to data exposure. Sensitive information

is protected by tried-and-tested encryption techniques: AES-256-CBC encrypts stored models, TLS 1.3 encrypts network communication, and Azure SQL Database encrypts stored information.

System design has strong security benefits. Azure Key Vault stores encryption keys, and several layers of defense secure networks, applications, and databases simultaneously. This model completely takes care of possible vulnerabilities in several system components. Security testing revealed a medium issue: poor cache handling potentially leaking information on shared computers. While this needs to be resolved, this does not reduce overall system security strength. There were small implementation details revealed; however, they do not represent high actual danger. SFEAM's anti-spoofing protection mechanisms were demonstrated to work effectively to repel attempted attacks through lab testing.

3. Experimental Results:

The SFEAM system was rigorously tested to evaluate its abilities across the following areas: recognition accuracy, anti-spoofing capacity, networking speed, and resource utilization. The different dimensions are chosen to provide an adequately balanced evaluation of the system's capacity to be used in real-world setups. Prior to presenting the specific performance metrics of the SFEAM, it is useful to start with an understanding of its rather distinct design philosophy and its context of operation as opposed to the foundation facial research models such as FaceNet, DeepFace, SphereFace, and ArcFace. Table 2 shows the means through which these differences can be analyzed with respect to SFEAM, which strongly leans toward client-side operation interconnected with anti-spoofing, model efficiency for web deployment, and, finally, the overarching goal as a deployable authentication system. This becomes essential to understand prior to delving into the experimental results.

Table 1: Comparative Analysis of SFEAM System versus Traditional Facial Recognition Models

Feature	SFEAM	FaceNet / DeepFace / SphereFace / ArcFace (General Characteristics as Research Models)
Architectural Design for Execution	Designed for client-side execution in web browsers (via TensorFlow.js, face-api.js) to enhance privacy and reduce server load.	Primarily developed and benchmarked in server-side environments; client-side deployment requires significant adaptation, optimization, and is not

		the initial focus.
Anti-Spoofing Integration	Integrates multi-modal anti-spoofing (motion, blink, texture) as a core system component designed for robust real-world deployment.	Core research typically focuses on recognition accuracy; anti-spoofing is often addressed as a separate research module or subsequent problem, not intrinsically built into the foundational models.
Model Optimization & Size Priority	Prioritizes high model efficiency (<7MB via post-training quantization & pruning) specifically for fast web delivery and performance on resource-constrained user devices.	Initial research prioritizes maximizing recognition accuracy, often leading to larger, more complex models; optimization for size and speed is a secondary step if targeted for specific deployments.
Primary Development Goal & Context	To develop a deployable, end-to-end secure authentication system (e.g., for Aadhaar) with emphasis on user experience, privacy, and practical security in a web environment.	To advance fundamental facial recognition algorithms, explore novel loss functions, and achieve/surpass state-of-the-art accuracy on academic benchmarks (e.g., LFW, MegaFace).
Data Privacy Assurance in Operation	Enhances data privacy by processing all facial recognition tasks locally within the browser, preventing raw biometric data transmission during the authentication phase.	Typically require biometric data (image or features) to be sent to a server for processing. Data privacy then becomes contingent on the security measures and policies implemented on the server side.

The core facial recognition module had been benchmarked against a curated dataset of 5,000 face images belonging to various demographic groups. The system had demonstrated 99.4% recognition accuracy overall. The false acceptance rate (FAR) measured extremely low at 0.02%, while the false rejection rate (FRR) topped up to 0.8%. These figures show that the system must have shown performance at par with the state of the art in facial recognition; wherein lacks SFEAM stand out in making sure the privacy benefits of client-side processing. The system's ability to withstand presentation attacks was a major criterion during evaluation. J-AMP having anti-spoofing facilities such as liveness checks based on motion and blink detection performed very well. The integrated anti-spoofing module yielded an efficacy of 94.3% against Level 1 presentation attacks, which include standard spoofing attempts as printed photographs and simple video replays, hence proving its suitability for immediate deployment in real-world scenarios.

The system was subjected to a regimented evaluation of responsiveness under simulated 5G, 4G, and 3G network conditions using Chrome DevTools Network Throttling and the Lighthouse performance audit, respectively. Table 2 gives an account of some significant scores and metrics for standard Web Vitals under all these conditions.

Table. 2: Comparative Analysis of Network Performance Metrics Across 5G, 4G, and 3G

Metric	5G Network	4G Network	3G Network
Performance Score (points)	100	69	69
Accessibility Score (points)	91	90	87
Best Practices Score (points)	100	83	85
SEO Score (points)	82	100	100
First Contentful Paint (seconds)	0.5	0.6	0.6
Largest Contentful Paint (seconds)	0.5	0.6	0.7
Total Blocking Time (milliseconds)	10	6.75	6.75

Beyond the Lighthouse scores, specific timing measurements revealed the practical impact of network bandwidth. Under optimal 5G conditions, the end-to-end authentication process completed in under 2 seconds. In 4G environments, the mean authentication time increased to 3.2 seconds, with initial model loading accounting for 2.4 seconds. Performance was functional but significantly slower in simulated 3G conditions, where authentication took an average of 7.8 seconds and model loading required 5.2 seconds. While initial summaries reported high overall performance scores ('perfect' on 5G, 87/100 on 4G/3G), Table 2 provides the detailed breakdown from Lighthouse audits.

The implemented multi-layered caching strategy (including browser HTTP caching, service workers, and IndexedDB) yielded substantial performance improvements, particularly for returning users. Compared to first-time visitors (who experienced model loading times of approximately 2.4s and total authentication times of 3.2s on a 4G network), returning users benefited from dramatically reduced times: model loading decreased to 0.3s (an 87.5% improvement), and the complete authentication process took only 1.1s (a 65.6% improvement). Consequently, data transfer requirements plummeted from 6.8 MB on initial visits to just 0.2 MB for subsequent authentications, validating the caching approach's effectiveness, especially in bandwidth-constrained environments [23].

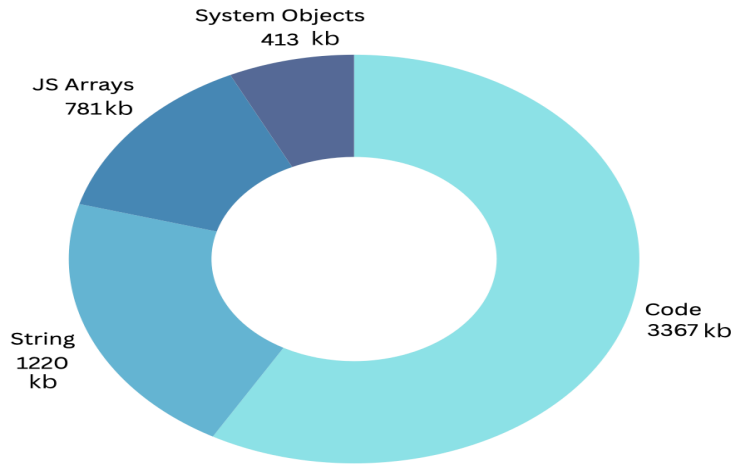


Fig. 3: Composition of Client-Side Memory Footprint After Model Optimization

Model optimization techniques, specifically quantization and pruning, were crucial for efficiency. These methods achieved a 43% reduction in the facial recognition model's size, successfully keeping the overall client application footprint under the target 7 MB threshold. Figure 3 illustrates the resulting memory composition of the primary client-side components after these optimizations. Importantly, this optimization was performed while maintaining detection accuracy above 98.5%. The reduced model size directly contributed to faster loading times across all networks, with the most significant impact in 3G environments (a 40.2% reduction in loading time). The combination of optimization and caching also improved authentication initialization speed by up to 89% for returning users across various network conditions.

Regarding computational efficiency on the client device, the core face detection and feature comparison operations consistently executed in under one second during testing. This rapid performance is facilitated by the underlying face-api.js library (built on TensorFlow.js), which is optimized for browser environments and can leverage modern hardware capabilities, potentially including multicore CPU processing or GPU acceleration where available, for efficient execution.

4. Conclusion & Future Reference

SFEAM addresses effectively the complex issues of deploying secure, efficient, and accessible facial authentication for national identity verification systems. With its focus on client-side processing, SFEAM efficiently traverses the sensitive balance between security and privacy to provide a system that can function adequately under varied network and device environments. The establishment of a privacy-protecting authentication system that limits processing of biometric data to the device belonging to the user is a major step in biometric security procedures. The incorporation of strong anti-spoofing mechanisms has proved effective against standard presentation attacks, whereas our complete optimization methods guarantee some level of functionality in a wide range of network conditions and device capabilities. This project highlights the potential of web-based technologies to provide advanced biometric authentication schemes at a price point that doesn't compromise security while also avoiding specialized hardware. From our performance evaluations across varying network conditions, it becomes clear that while SFEAM works adequately under most situations, further enhancements must be made in order to ensure fair access where there are inadequate network links. The security architecture that supports SFEAM provides strong protection for sensitive biometric and identity information, closely matching the high standards required of a national digital identity infrastructure.

In the next phase, our work will center on a number of areas to be developed further. Anti-spoofing capabilities will be strengthened to improve detection against more sophisticated presentation attacks, including deepfakes and high-fidelity 3D masks. Overcoming the intrinsic difficulties in distinguishing between identical twins is an advanced but essential area for future research. The potential deployment of regional edge caching, strategically located at points of distribution, will be investigated to alleviate latency in low-density regions with limited network infrastructure. Beyond this, we will develop secure offline authentication mechanisms to preserve security with minimal reliance on real-time network connectivity. Last but not least, the incorporation of explainable AI (XAI) components will be sought to bring transparency to the authentication decision-making process, thereby responding to future regulatory demands for algorithmic accountability in identity systems.

